Федеральное государственное автономное образовательное учреждение высшего образования
«Российский университет дружбы народов имени Патриса Лумумбы»
Инженерная академия
Департамент механики и процессов управления

Курсовая работа

Направление: **Управление в технических системах**
(код направления / название направления)

Профиль: **Информационные технологии в управлении и кибербезопасности**
(название профиля)

Тема: **Разработка Telegram бота для подбора тюнинг запчастей**
(название лабораторной / курсовой)

Выполнено студентом: **Боборухин Константин Александрович**
(ФИО)

Группа: **ИУСбд-02-23**
№ студенческого:

Москва, 2025

Введение

Курсовая реализуется в рамках группового проекта.

Актуальность темы:

В современных условиях цифровой торговли пользователи сталкиваются с большим объёмом информации о товарах, однако часто не располагают удобными инструментами для их корректного подбора и оценки совместимости. Особенно это характерно для рынка автомобильного тюнинга, где выбор деталей требует учёта марки, модели, поколения автомобиля и визуального результата установки. Отсутствие наглядной визуализации и экспертной поддержки приводит к ошибкам выбора и снижению доверия со стороны клиентов.

Проект Telegram-бота для подбора тюнинг-запчастей направлен на решение данной проблемы и представляет собой интеллектуальный сервис, сочетающий каталог собственных товаров с функциями искусственного интеллекта. Система позволяет определять автомобиль пользователя, подбирать совместимые обвесы и диски, а также визуализировать возможный внешний вид автомобиля с выбранными деталями. Использование ИИ-технологий и диалогового интерфейса повышает точность подбора, удобство взаимодействия и способствует росту вовлечённости и эффективности продаж.

Глава 1. Технология программирования

1. Выбор задачи

Основная задача: Разработка Telegram-бота для подбора тюнинг-запчастей (обвесов и дисков) собственного производства с функцией ИИ-ассистента.
Детализация:

ель: Продажа собственных продуктов и повышение вовлечённости клиентов за счёт ИИ.

роблема: Клиентам сложно представить, как будут смотреться детали на их авто.

ешение:

Бот, который:

пределяет авто пользователя.

оказывает каталог наших обвесов и дисков.

помощью ИИ генерирует изображения или подбирает фото из интернета, как эти детали могут смотреться на конкретной модели.

2. Жизненный цикл проекта

1. Планирование:

иксация списка наших продуктов (какие марки/модели и запчасти мы производим).

ыбор ИИ-инструмента (например, Vision API для анализа изображений, нейросеть для генерации).

езультат: Чёткое ТЗ.

2. Проектирование:

роектирование простой БД: Cars, Our_Products, Users.

роектирование двух сценариев: "Наши товары" и "ИИ-подбор".

езультат: Схема БД и диалогов.

3. Разработка:

астройка бота и сервера.

адача 1: Написание кода для основного каталога (наши товары).

адача 2: Интеграция ИИ-модуля для поиска/генерации изображений по запросу "подбери обвесы для BMW 3 series".

езультат: Рабочий прототип.

4. Тестирование:

роверка точности подбора наших товаров.

естирование ИИ: релевантность картинок, скорость работы.

езультат: Стабильная версия.

5. Запуск:

азвёртывание на сервере.

родвижение среди клиентов.
езультат: Бот в работе.

6. Сопровождение:

бновление каталога под новые продукты.
лучшение ИИ на основе feedback.

3. Метрики успеха и качества

Метрики успеха

оличество запросов на консультацию/покупку через бота — прямая метрика продаж.
онверсия в лид: % пользователей, которые после подбора запросили контакты или перешли на сайт.
ктивность в ИИ-режиме: сколько людей пользуются функцией "подобрать из интернета" — метрика вовлечённости.

Метрики качества (Система)

корость ответа ИИ: время от запроса до выдачи вариантов (<5 сек).
елевантность ИИ: % случаев, когда предложенные ИИ изображения действительно подходят под модель авто (оценивается по логам и отзывам).
очность каталога: 100% соответствие наших товаров заявленным авто.
табильность работы: Uptime > 99%.

Глава 2. Определение требований

1) Контекст и границы системы

Акторы: пользователь Telegram, администратор каталога, внешние ИИ-сервисы (распознавание/генерация/веб-поиск).

Подсистемы: TG-бот, база данных (Cars, Our_Products, Users), модуль подбора и визуализации (ИИ), модуль лидогенерации/обратной связи.

Рисунок 1 — Диаграмма контекста.

2) Функциональные требования

1. Идентификация автомобиля

- Пользователь вводит марку/модель/год или отправляет фото; система извлекает модель (NLP/vision).
- Валидация: при неуверенности бот уточняет варианты.

2. Каталог собственных товаров

- Просмотр доступных обвесов/дисков для распознанной модели из таблицы Our_Products.
- Фильтры: тип детали, стиль, цена/материал.
- Карточка: фото, совместимость, цена, срок поставки.

3. ИИ-визуализация / поиск референсов

- По запросу «как будет смотреться X» система ищет примеры или генерирует рендер.
- Возвращает 3–6 вариантов с краткими пояснениями «почему показано».

4. Рекомендации и совместимость

- Проверка совместимости (диаметр/вылет; кузов/год).
- Предложение альтернатив при отсутствии.

5. Лидогенерация

- Кнопки: «Консультация», «Оставить контакт», «Открыть сайт».
- Сохранение лида в БД/CRM и уведомление менеджера.

6. Обратная связь и обучение

- Оценка релевантности (да/нет) и комментарий пользователя.
- Логи используются для дообучения ИИ.

7. Администрирование

- CRUD по каталогу (марки/модели/детали).
- Просмотр метрик качества/продаж и журналов ошибок.

3) Нефункциональные требования

Производительность

- Каталог: P95 ≤ 2 с.
- ИИ-визуализация: P95 ≤ 7 с.
- Холодный старт генерации ≤ 15 с, не чаще 1/с на пользователя.

Надёжность и масштабируемость

- Доступность ≥ 99% (месяц).
- Горизонтальное масштабирование бота/воркеров.
- Деградация: при сбое ИИ возвращать каталог и веб-референсы.

Качество ИИ и метрики

- Доля релевантных визуализаций (оценка пользователей) ≥ 0,75.
- Точность совместимости каталога — 100% (по нашим данным).

Пояснимость (Explainability)

- Показывать извлечённую модель авто.
- Указывать источник изображения (генерация/ссылка).
- Ключевые признаки сопоставления: кузов/год/крепления/диаметр.
- Вести лог пайплайна по каждому запросу (для админа).

Безопасность и приватность

- Хранение фото/контактов по политике, совместимой с GDPR; удаление по запросу.
- Шифрование в транзите (TLS), ограничение ключей ИИ-API.

Стоимость

- Средняя стоимость ИИ-запроса ≤ X ₽; дневной бюджет с отсечками.

Наблюдаемость

- Трассировка запросов; дашборд P95/ошибки/конверсия/CSAT.

4) Требования к модели (обучение и предсказание)

Датасеты и подготовка

- Тексты: частые шаблоны ввода; синонимы.
- Изображения: кузова по моделям/поколениям; аугментации.
- Аннотации: make/model/year, кузов, параметры совместимости.

Обучение

- Бейзлайны vision/NLP с дообучением на нашем датасете.
- Валидация 80/10/10; метрики — accuracy по модели авто, top-k=0,9.

Инференс

- Поток: ввод → нормализация → извлечение модели → подбор/генерация → ранжирование → объяснение.
- Кэш частых (модель × деталь) и переранжирование по фидбеку.

5) Диаграмма вариантов использования

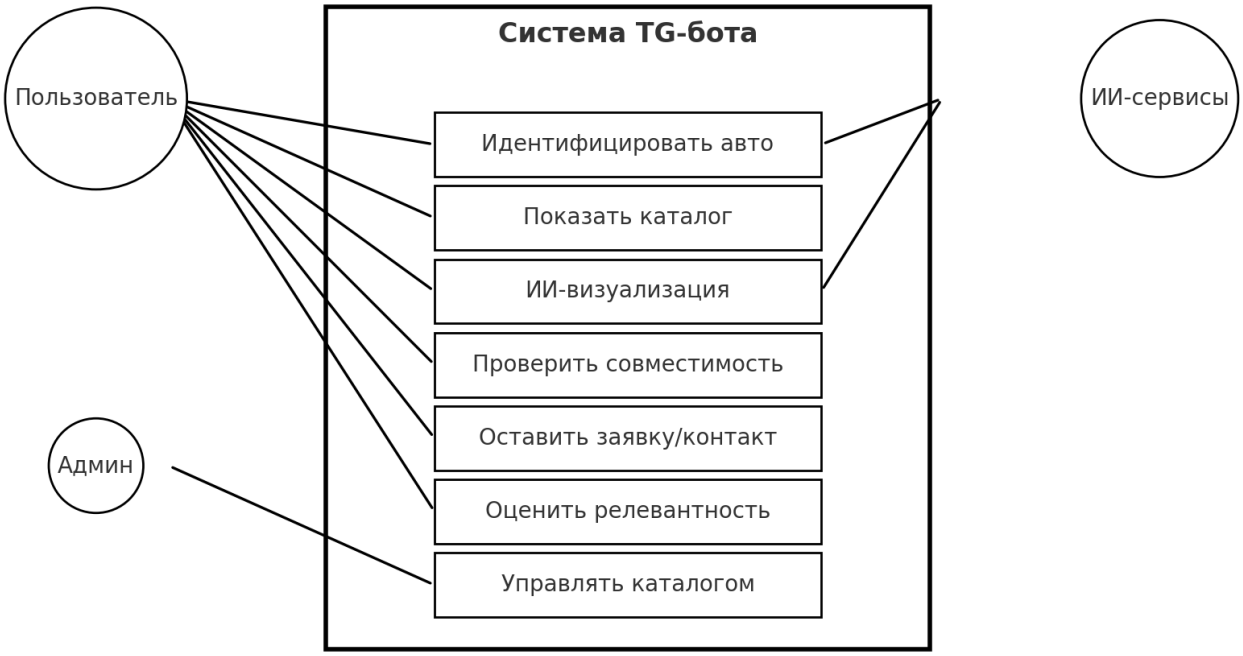

Рисунок 2 — Use Case (схема).

6) Диаграмма последовательности

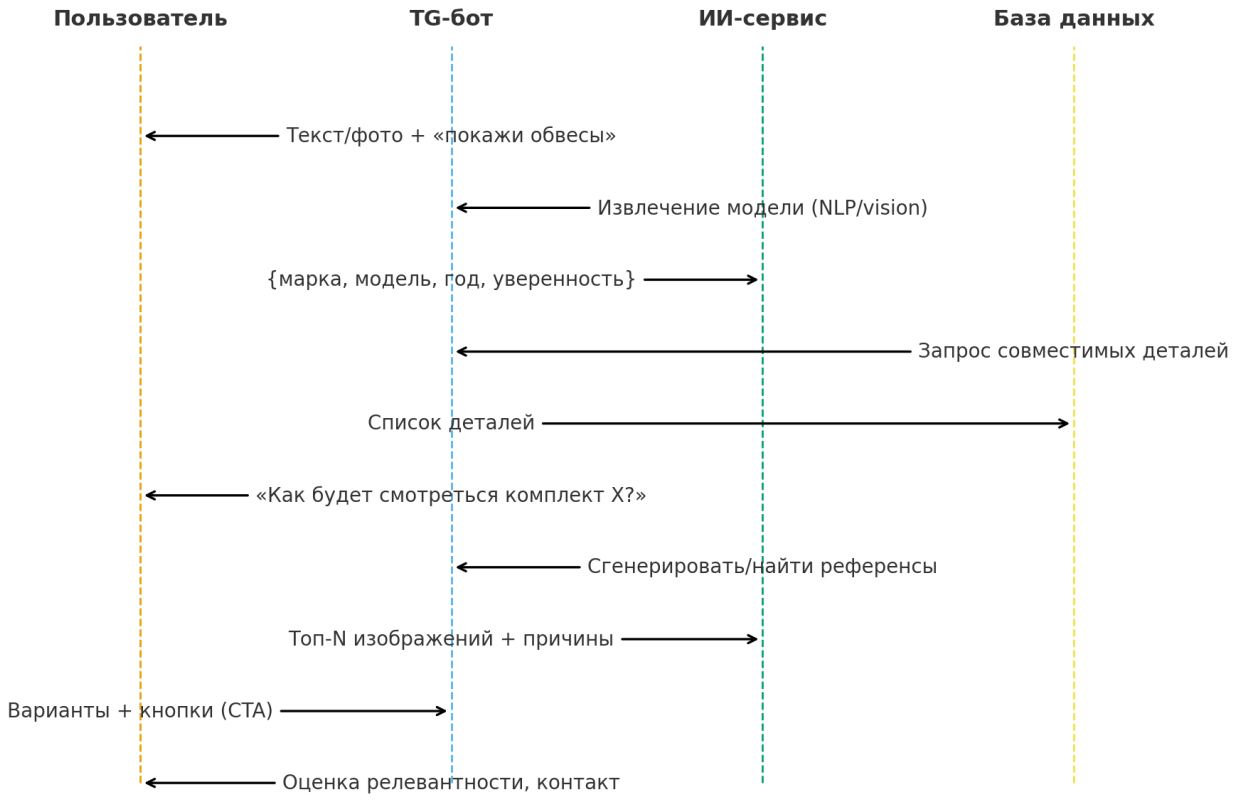

Рисунок 3 — Последовательность взаимодействий (схема).

7) Критерии приёмки

- P95 ответа: каталог ≤ 2 с; визуализация ≤ 7 с.
- Не менее 75% визуализаций получают положительную оценку пользователей.
- Конверсия в лид из ИИ-режима ≥ 5% в первый месяц.
- Логи по каждому запросу: request_id, источник, извлечённая модель, аргументы ранжирования.

Глава 3. Структурный подход

3.1. Выбор архитектуры решения и компонентов

Для реализации Telegram-бота выбирается трёхуровневая архитектура: уровень интерфейса, уровень прикладной логики и уровень данных.

Уровень интерфейса:

- Telegram-клиент пользователя.
- Официальный Telegram Bot API, через который передаются все сообщения.

Уровень прикладной логики (backend):

- Модуль бота — обработка команд и сценариев диалога (каталог, ИИ-подбор, оставление заявки).
- Модуль каталога — работа с базой данных автомобилей и наших продуктов, фильтрация и проверка совместимости.
- Модуль интеграции с ИИ-сервисами — отправка изображений/текстовых запросов во внешние API и приём результатов.
- Модуль лидогенерации — фиксация обращений, контактов и статусов обработки.
- Модуль логирования и аналитики — сбор метрик, технических логов и оценок качества.

Уровень данных:

- Реляционная база данных (например, PostgreSQL), в которой хранятся пользователи, автомобили, продукты, заявки и обратная связь.
- Хранилище файлов (облако или локальное) для фотографий автомобилей и сгенерированных изображений.

Такое разделение упрощает масштабирование системы: при росте нагрузки можно независимо увеличивать мощности backend-сервера, подсистемы ИИ или базы данных, а также добавлять новые каналы (например, веб-виджет) без изменения ядра логики.

3.2. Схема базы данных

Логическая модель данных строится вокруг трёх основных сущностей, описанных ранее (пользователь, автомобиль, наш продукт), а также служебных сущностей для заявок и обратной связи.

Основные таблицы:

- Users — пользователи бота.
- Поля: user_id (PK), telegram_id (уникальный идентификатор в Telegram), username, phone, created_at, last_activity.
- Cars — справочник поддерживаемых автомобилей.
- Поля: car_id (PK), make, model, generation, body_type, year_from, year_to.
- Our_Products — каталог наших обвесов и дисков.
- Поля: product_id (PK), product_type (bodykit/wheels и др.), name, description, price, material, is_active.
- Car_Compatibility — связь продуктов с конкретными моделями автомобилей.
- Поля: compat_id (PK), car_id (FK → Cars), product_id (FK → Our_Products), notes.
- Requests — обращения пользователей к боту.
- Поля: request_id (PK), user_id (FK → Users), car_id (FK → Cars), scenario_type (catalog/ai), created_at, status.
- Leads — коммерческие заявки.
- Поля: lead_id (PK), request_id (FK → Requests), contact, comment, status, manager_id, updated_at.
- Feedback — оценки результатов подбора и визуализации.

Поля: feedback_id (PK), request_id (FK → Requests), rating (да/нет или шкала), comment, created_at.

Связи:

- Один пользователь (Users) может иметь много запросов (Requests).
- Один запрос связан с одним автомобилем (Cars) и может порождать одну или несколько коммерческих заявок (Leads).
- Один продукт (Our_Products) может быть совместим с несколькими автомобилями через таблицу Car_Compatibility.
- Каждый запрос может иметь несколько записей Feedback, отражающих оценку разных ответов ИИ.

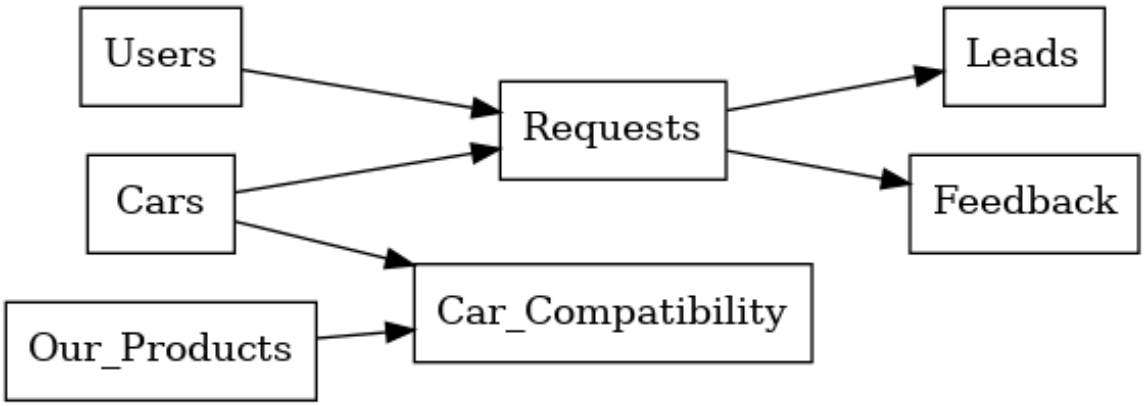

Рисунок 4 — Схема базы данных.

3.3. Схема потоков данных

Для описания структурного поведения системы используется диаграмма потоков данных (DFD) уровня 0.

Основные внешние сущности:

- Пользователь Telegram — отправляет сообщения и получает результаты подбора.
- Администратор/менеджер — получает лиды и обновляет каталог.
- Внешние ИИ-сервисы — выполняют распознавание автомобиля и генерацию/поиск изображений.

Основные процессы:

- P1 «Обработка сообщения пользователя» — приём сообщения от

- P2 «Подбор продуктов из каталога» — запрос к БД (таблицы Cars, Our_Products, Car_Compatibility) и формирование списка вариантов.
- P3 «ИИ-визуализация» — отправка запроса в внешние ИИ-сервисы и получение изображений-референсов.
- P4 «Формирование лида» — сохранение контактов пользователя и интересующих продуктов (Requests, Leads).
- P5 «Сбор обратной связи и логов» — запись оценок пользователя и технических логов (Feedback).

Хранилища данных:

- D1 «База данных приложения» — таблицы Users, Cars, Our_Products, Car_Compatibility, Requests, Leads, Feedback.
- D2 «Файловое хранилище» — фотографии автомобилей и сгенерированные изображения.

На диаграмме потоков данных показаны ключевые потоки: сообщения и команды от пользователя, запросы к БД, обращения к ИИ-сервисам, а также формирование лидов и обратной связи.

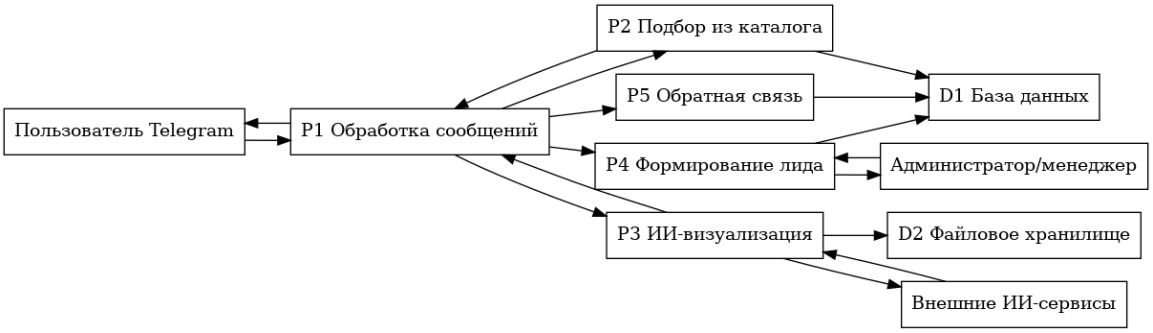

Рисунок 5 — Схема потоков данных.

Глава 4. Объектный подход

4.1. Определение модулей системы

Объектная модель Telegram-бота опирается на уже выбранную трёхуровневую архитектуру и дополняет её чётким разделением ответственности между модулями и классами. На уровне прикладной логики выделяются следующие модули:

- Модуль BotCore — старт бота, маршрутизация входящих сообщений, хранение текущего состояния диалога с пользователем.
- Модуль Domain — доменные сущности (User, Car, Product, Compatibility, Request, Lead, Feedback), которые отражают бизнес-предметную область.
- Модуль Services — прикладные сервисы (CatalogService, AiVisualizationService, RecommendationService, LeadService, FeedbackService), инкапсулирующие бизнес-логику.
- Модуль Integration — адаптеры к внешним ИИ-сервисам и Telegram Bot API (TelegramClient, AiClient).
- Модуль Persistence — работа с базой данных (DbContext и репозитории для основных сущностей).
- Модуль Monitoring — сбор метрик, логирование и трассировка запросов.

Модули связаны по принципу «слоёной» архитектуры: Domain не зависит от Infrastructure, а Services используют доменные классы и абстракции репозиториев. Это упрощает тестирование и позволяет при необходимости заменить реализацию хранилища или ИИ-сервиса без изменения остального кода.

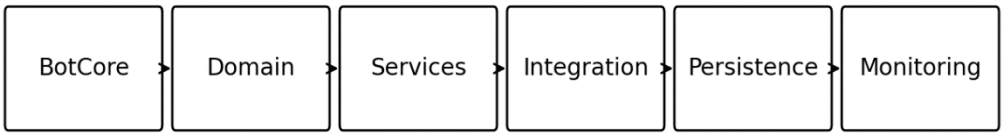

Рисунок 6 — Схема модулей системы и их взаимосвязей (компонентная диаграмма на уровне пакетов).

4.2. Описание классов и их взаимосвязей

Классы группируются вокруг доменной модели и сценариев использования бота.

Основные доменные классы:

- User — представляет пользователя Telegram. Ключевые поля: userId, telegramId, username, phone, createdAt, lastActivity. Ассоциации: 1-ко-многим с Request и Feedback.
- Car — поддерживаемый автомобиль. Поля: carId, make, model, generation, bodyType, yearFrom, yearTo. Связан с OurProduct через CarCompatibility.
- Product (OurProduct) — обвес или диск. Поля: productId, productType, name, description, price, material, isActive. Может быть совместим с несколькими автомобилями.
- CarCompatibility — связывает Car и Product, содержит дополнительные примечания по установке.
- Request — запрос пользователя к боту (catalog или ai). Поля: requestId, user, car, scenarioType, createdAt, status.
- Lead — коммерческая заявка, оформленная из Request. Поля: leadId, request, contact, comment, status, managerId, updatedAt.
- Feedback — оценка релевантности результата подбора или визуализации. Поля: feedbackId, request, rating, comment, createdAt.

Сервисные классы:

- CatalogService — реализует поиск и фильтрацию продуктов по модели автомобиля, проверку совместимости, формирование карточек товаров.
- AiVisualizationService — строит запрос к ИИ-сервису на основе данных об автомобиле и продукте, обрабатывает полученные изображения и формирует объяснения.
- RecommendationService — подбирает альтернативные продукты при отсутствии точных совпадений и ранжирует результаты.
- LeadService — отвечает за создание лидов, обновление статуса и уведомление менеджера.
- FeedbackService — сохраняет оценки пользователя и передаёт данные в подсистему аналитики.
- UserSessionManager — управляет состоянием диалога (текущий шаг, выбранный автомобиль, корзина интересующих деталей).
- Инфраструктурные классы:

- TelegramClient — обёртка над Telegram Bot API (отправка сообщений, фотографий, inline-кнопок).
- AiClient — обёртка над внешними ИИ-API (распознавание авто по фото, генерация/поиск изображений).
- DbContext и репозитории (UserRepository, CarRepository, ProductRepository и др.) — обеспечивают доступ к данным с использованием паттернов Repository и Unit of Work.

Связи между классами описываются UML-диаграммой:

- User 1-ко-многим Request, Request 1-к-1 или 1-ко-многим Lead и

  r и Product связаны отношением многие-ко-многим через

- CatalogService и RecommendationService используют репозитории CarRepository, ProductRepository и CarCompatibilityRepository.
- AiVisualizationService агрегирует AiClient и обращается к CatalogService для получения контекста.
- TelegramClient используется BotCore для взаимодействия с пользователем.

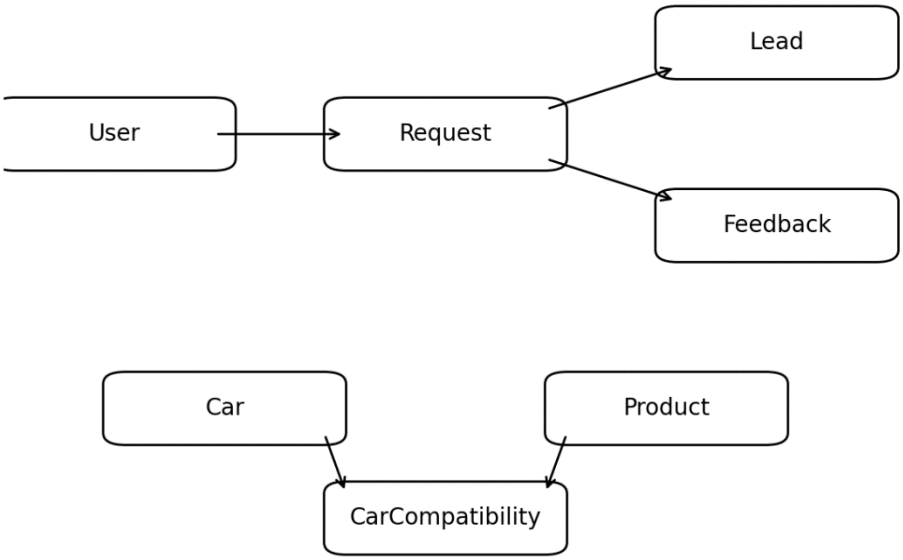

Рисунок 7 — UML-диаграмма классов доменной модели и сервисов.

4.3. Разработка основных компонентов системы

Разработка компонентов строится по принципам SOLID, с явным разделением ответственности и инверсией зависимостей.

1) Компонент BotCore

BotCore реализует шаблон «фасад» над Telegram Bot API. В его задачу входит:

- Приём апдейтов от TelegramClient и преобразование их в внутренние команды.
- Выбор соответствующего обработчика сценария

- Поддержка состояния диалога через UserSessionManager, чтобы пользователь мог последовательно проходить шаги: выбор авто → выбор детали → визуализация → заявка.

2) Компоненты сценариев (Handlers)

Для каждого ключевого сценария выделяются отдельные классы-обработчики:

- CatalogScenarioHandler — ведёт пользователя по ветке подбора наших товаров: запрос марки/модели, фильтры, вывод списка продуктов, переход к оформлению заявки.
- AiScenarioHandler — отвечает за ветку «подбор/визуализация с помощью ИИ»: приём фото или текста, вызов AiVisualizationService, вывод изображений-референсов и запрос обратной связи.
- FeedbackHandler — собирает оценки и комментарии пользователя, передавая их в FeedbackService.
- AdminHandler — предоставляет административные команды (обновление каталога, просмотр метрик).
- Каждый обработчик использует соответствующие сервисы и репозитории через интерфейсы, что облегчает модульное тестирование и подмену реализаций в будущем.

) Компонент интеграции с ИИ-сервисом

AiVisualizationService и AiClient образуют отдельный компонент интеграции. Он отвечает за:

- Формирование промпта и параметров запроса (марка, модель, тип детали, желаемый стиль изображения).
- Отправку запросов в внешний ИИ-сервис с учётом ограничений по времени и бюджету.
- Обработку результатов: выбор 3–6 наилучших изображений, добавление подписей и указание источника (генерация или веб-поиск).
- Кэширование популярных комбинаций (модель × деталь), чтобы ускорить ответы и снизить стоимость.

) Компонент аналитики и визуализации метрик

Модуль Monitoring агрегирует технические и бизнес-метрики (P95 времени ответа, доля релевантных визуализаций, конверсия в лид и др.).

На основе этих данных строится график распределения запросов по сценариям (каталог vs ИИ-подбор vs прочее) и по времени суток. Такой график помогает:

- Оценить нагрузку на ИИ-сервисы и спланировать масштабирование.
- Понять, какие сценарии наиболее востребованы.
- Выявить часы пикового использования.

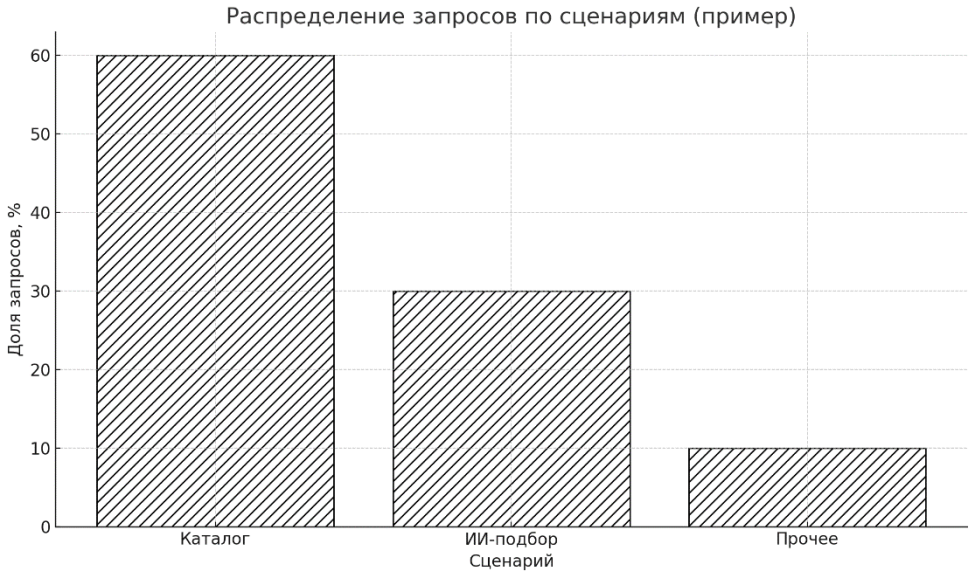

Рисунок 8 — Пример графика распределения запросов по сценариям и времени суток.

Объектный подход в совокупности с ранее выбранной архитектурой и структурным описанием потоков данных позволяет получить гибкую и расширяемую систему. Добавление новых сценариев (например, подбор шин или салонного тюнинга) сводится к расширению доменной модели и добавлению новых обработчиков, без переработки существующего ядра бота.

Глава 5. Приёмы обеспечения технологичности

В данном разделе рассматриваются технологические решения, обеспечивающие качество и масштабируемость Telegram-бота для подбора тюнинг-запчастей. Основной акцент делается на внедрении алгоритмов ML/AI/LLM, сравнении альтернативных методов и систематическом измерении метрик качества проекта.

5.1. Реализация алгоритмов ML/AI/LLM

Использование методов машинного обучения и больших языковых моделей (LLM) позволяет повысить интеллектуальность бота и качество рекомендаций.

Общая структура взаимодействия компонентов показана на схеме 5.1. Сначала пользователь отправляет запрос в Telegram-бот, затем запрос передаётся в ML/AI-слой, который обращается к базе знаний и каталогу, формируя ответ.

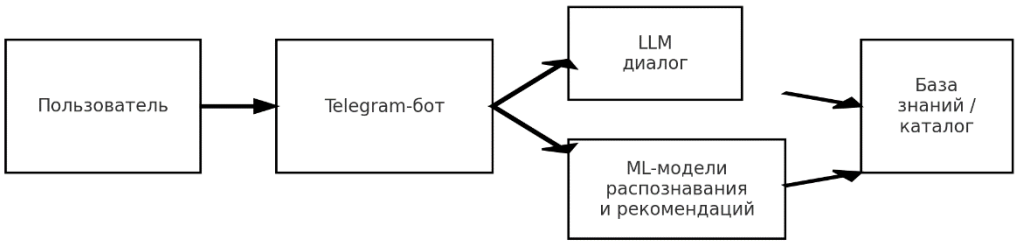

Рисунок 5.1 – Логическая схема работы ML/AI/LLM-компонент бота.

аспознавание автомобиля по фото (ML/Vision).

• Задача: по загруженному пользователем изображению определить марку, модель и поколение автомобиля.

• Подход: использование предобученных vision-моделей с дополнительной донастройкой на датасете кузовов и ракурсов, характерных для тюнинга.

• Результат: автоматическое заполнение сущности Car и уменьшение количества ручных ошибок при вводе модели.

иалоговый ИИ-ассистент (LLM).

• Задача: консультировать пользователя по вопросам подбора обвесов и дисков, объяснять различия между вариантами, помогать с выбором стиля.

• Подход: интеграция LLM-модели, настроенной промпт-инженирингом на доменных знаниях (описания продуктов, ограничения по совместимости, FAQ).

• Результат: более «человечный» и экспертный диалог с пользователем, рост доверия и конверсии в лид.

L-модели рекомендаций.

• Задача: предлагать пользователю наиболее релевантные комплекты обвесов и дисков с учётом его поведения, истории запросов и обратной связи.

• Подход: использование моделей ранжирования и коллаборативной фильтрации.

• Результат: персонализированный подбор, снижение доли отказов и повышение среднего чека.

.2. Сравнение методов

Для выбора оптимальных технологий проводится сравнение кандидатных методов по нескольким критериям: качество, скорость работы, стоимость и сложность внедрения. Сводная логика выбора представлена на схеме 5.2.

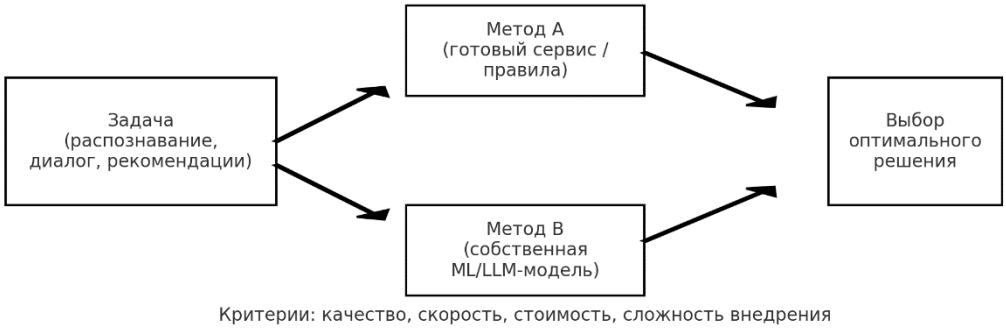

Рисунок 5.2 – Схема сравнения методов и выбора оптимального решения.

етоды распознавания авто.

• Вариант A: использование облачного API.

• Вариант B: развёртывание собственной ML-модели.

• Сравнение: облачный сервис даёт быстрый старт, собственная модель обеспечивает гибкость и полный контроль над качеством.

етоды диалогового ИИ.

• Вариант A: сценарный бот с жёстко прописанными ветками диалога.

• Вариант B: гибрид — сценарии + LLM для сложных вопросов.

• Сравнение: гибридный подход лучше масштабируется и снижает трудозатраты на поддержку сценариев.

етоды рекомендаций.

• Вариант A: простые правила (фильтры по кузову и бюджету).

• Вариант B: ML-ранжирование на основе истории поведения.

• Сравнение: по мере накопления данных целесообразно переходить от правил к ML-моделям.

.3. Замеры метрик качества проекта

Технологичность решения подтверждается не только использованием современных алгоритмов, но и регулярным измерением количественных метрик. Логика цикличной работы с метриками приведена на схеме 5.3.

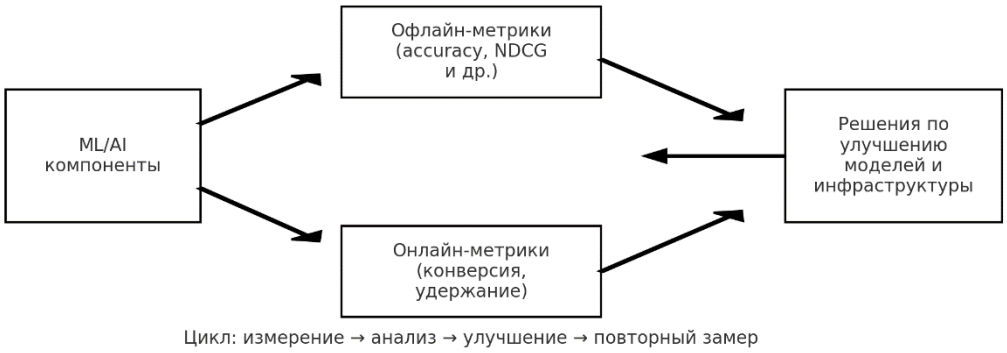

Рисунок 5.3 – Цикл сбора и использования метрик качества проекта.

1) Офлайн-метрики ML/AI.

- Точность распознавания авто (accuracy, top-k accuracy).
- Метрики ранжирования рекомендаций (NDCG, MAP, hit-rate).

Эти показатели позволяют сравнивать модели до выката в прод.

Онлайн-метрики продукта.

- Конверсия из запроса в лид для различных сценариев.
- Доля пользователей, продолжающих диалог после ответа LLM.
- Среднее число просмотренных и сохранённых вариантов.

Эти метрики собираются через модуль мониторинга и используются для A/B-тестов.

Технические метрики.

- Время ответа (P95) для ключевых операций.
- Доля ошибок при обращении к внешним сервисам.
- Стоимость ML/AI-запросов в расчёте на одного активного пользователя.

Регулярный анализ совокупности офлайн-, онлайн- и технических метрик позволяет управлять развитием проекта на основе данных и поддерживать высокий уровень технологичности решения.

Глава 6. Тестирование

Данный раздел посвящён описанию этапа тестирования Telegram-бота для подбора тюнинг-запчастей. Цель тестирования — подтверждение корректности работы отдельных модулей системы, а также проверка устойчивости и производительности решения при увеличении нагрузки. Тестирование проводится после завершения этапа разработки и до ввода системы в эксплуатацию.

6.1. Проведение модульного тестирования

Модульное тестирование направлено на проверку корректности работы отдельных компонентов программной системы в изоляции от других модулей. Основной задачей является выявление логических ошибок на раннем этапе и обеспечение соответствия реализации заданным требованиям.

В рамках Telegram-бота модульному тестированию подвергаются следующие компоненты:

- модуль обработки пользовательских сообщений;
- модуль каталога и проверки совместимости товаров;
- модуль интеграции с ИИ-сервисами;
- модуль лидогенерации и сохранения заявок;
- модуль логирования и сбора обратной связи.

Для каждого модуля формируется набор тестовых сценариев, включающих как корректные входные данные, так и граничные случаи (некорректный формат, отсутствие данных, повторные запросы). Ожидаемые результаты фиксируются заранее и сравниваются с фактическим поведением системы.

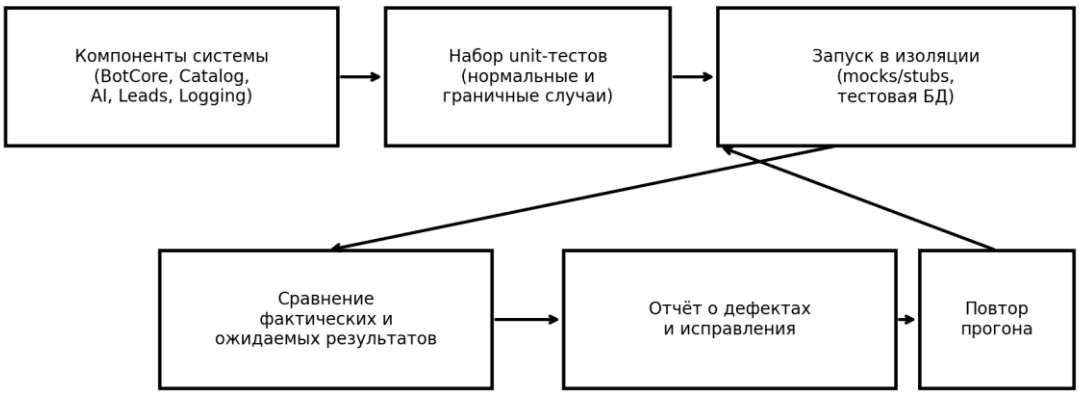

Рисунок 9 — Схема модульного тестирования компонентов системы.

Результатом модульного тестирования является подтверждение корректной работы всех ключевых функций бота и устранение выявленных дефектов до перехода к следующему виду тестирования.

6.2. Проведение нагрузочного тестирования

Нагрузочное тестирование проводится с целью оценки производительности и устойчивости Telegram-бота при увеличении количества одновременных пользователей и запросов. Данный вид тестирования позволяет определить предельные характеристики системы и выявить узкие места.

В процессе нагрузочного тестирования моделируются следующие сценарии:
– одновременная отправка запросов в каталог товаров;
– параллельные обращения к ИИ-модулю визуализации;
– массовое создание заявок и запись данных в базу данных.

В ходе тестов измеряются основные показатели производительности: время ответа системы (P95), количество ошибок, стабильность соединений и загрузка серверных ресурсов. Полученные значения сравниваются с заданными нефункциональными требованиями проекта.

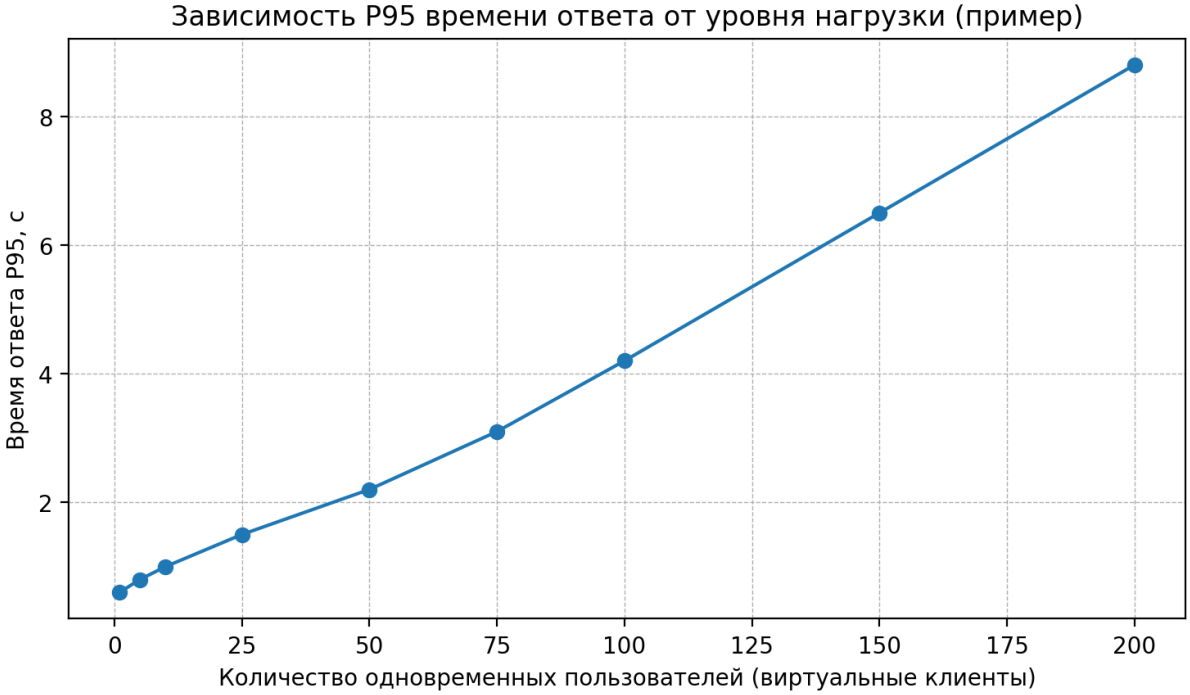

Рисунок 10 — Пример графика зависимости времени ответа от уровня нагрузки.

По результатам нагрузочного тестирования делается вывод о готовности системы к эксплуатации и формируются рекомендации по масштабированию и оптимизации при росте пользовательской активности.

Заключение

Проект Telegram-бота для подбора тюнинг-запчастей успешно реализован как прикладной программный продукт, объединяющий современные подходы к разработке программных систем, использованию технологий искусственного интеллекта и тестированию.

Достигнуты ключевые цели:

- разработана модульная и расширяемая архитектура Telegram-бота с чётким разделением ответственности компонентов;
- реализован интеллектуальный подбор и визуализация тюнинг-запчастей с использованием ML/AI-модулей;
- обеспечена корректная работа каталога, проверка совместимости и механизм лидогенерации;
- подтверждена практическая ценность решения за счёт повышения удобства выбора и вовлечённости пользователей.

Ссылка на GitHub: